

Likelihood and log probabilities

The probability of the data as a function of the parameter, why we take its log, and where it peaks.

THE QUESTION

Which parameter values make the data we saw most probable, and how do we compute that without the numbers vanishing?

THE INTUITION

Fix the data and slide the parameter: $p(\text{data} \mid \theta)$ as a function of θ is the likelihood, and its peak is the maximum likelihood estimate. For i.i.d. data it is a product of many small numbers, so we work with its log, a sum.

KEY TAKEAWAY

$\hat{\theta} = \arg \max_{\theta} \sum_i \log p(x_i \mid \theta)$. Maximising the log-likelihood is minimising the mean negative log-likelihood, the usual loss.

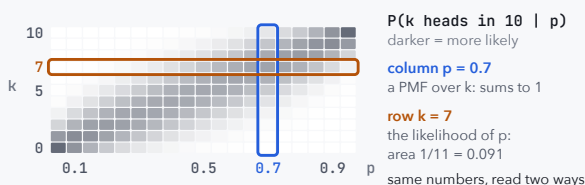
data x, fixed $\rightarrow L(\theta) = p(x \mid \theta) \rightarrow$ log L: a sum over points $\rightarrow$ slope = 0, or steps $\rightarrow$ the MLE $\rightarrow$ NLL = loss

1 The likelihood scores each parameter on fixed data



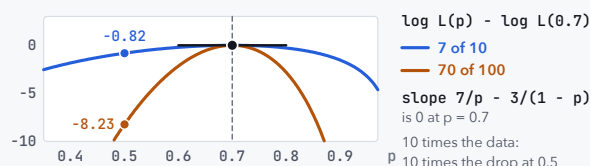
A coin shows 7 heads in 10 flips. If the heads chance is p , that has probability $L(p) = \binom{10}{7} p^7 (1-p)^3 = 120 p^7 (1-p)^3$. The data stay fixed and p varies: $L(0.5) = 11.7\%$, $L(0.7) = 26.7\%$, $L(0.9) = 5.7\%$, so 0.7 explains these flips best of the three.

2 A likelihood is not a distribution over the parameter



A column of $P(k \mid p)$ is a PMF: it sums to 1 over k . A row fixes the data and varies p , and nothing makes it add to 1: the row $k = 7$ has area $\int_0^1 120 p^7 (1-p)^3 dp = 1/11$. So $L(0.7) = 0.267$ is not "the chance that p is 0.7"; it ranks values of p on these data.

3 The log has the same peak: set its slope to 0



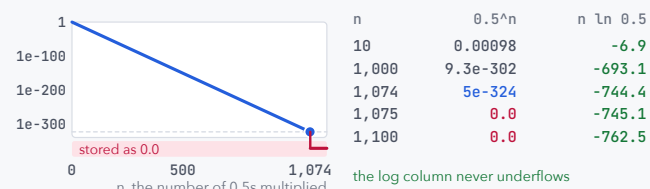
Log is increasing, so L and $\ell(p) = \log L(p)$ peak at the same p . Here $\ell(p) = \log 120 + 7 \log p + 3 \log(1-p)$, and $\ell'(p) = 7/p - 3/(1-p) = 0$ gives $\hat{p} = 7/10$. With k heads in n , the maximum likelihood estimate (MLE) is $\hat{p} = k/n$. 70 of 100 has the same peak and a drop 10 times deeper: more data pin p down harder.

4 i.i.d. data: a product of probabilities, a sum of logs



For independent draws from one model, $L(\theta) = \prod_i p(x_i \mid \theta)$ and $\ell(\theta) = \sum_i \log p(x_i \mid \theta)$ (the joint distributions sheet). At $p = 0.7$ each head adds $\ln 0.7$ and each tail $\ln 0.3$. This order of flips has probability $0.7^7 0.3^3 = 0.00222$; the missing factor 120 does not move $\hat{p}$, nor does dividing by n to report the mean.

5 Long products of probabilities underflow



The smallest positive float64 is $0.5^{1074} \approx 5 \times 10^{-324}$; 0.5^{1075} rounds to 0.0. So the likelihood of 1,100 fair flips comes out as 0, and every p looks equally bad, while its log $1100 \ln 0.5 = -762.5$ is harmless. $\mathcal{N}(0, 1)$ at 17 has log density -145.42 ; the density 7×10^{-64} is 0.0 in float32, PyTorch's default. So distributions return `log_prob`.

6 Log-sum-exp adds probabilities in log space

A mixture density or a normalisation adds probabilities: $\log(e^a + e^b)$, where e^{-1000} is 0.0. Pull out the larger m first: $m + \log(e^{a-m} + e^{b-m})$, the log-sum-exp trick.

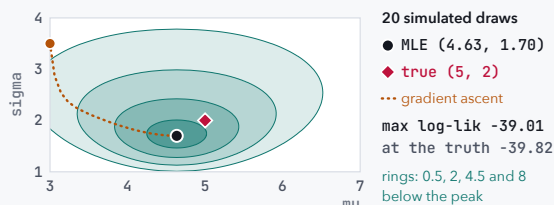
```
a, b = -1000.0, -1001.0 # log probs
np.log(np.exp(a) + np.exp(b)) # -inf
np.logaddexp(a, b) # -999.687
```

`torch.logsumexp` does it over an axis. A value outside the support has log probability $-\infty$ (PyTorch raises an error by default): one impossible point sends all of ℓ to $-\infty$.

Maximum likelihood in practice

Closed forms for the common families, a gradient loop for the rest, and the loss that comes out of it.

7 Normal MLE: the sample mean, and the SD divided by n



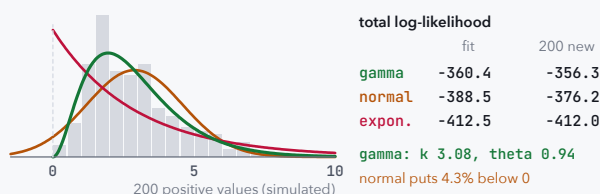
$\ell(\mu, \sigma) = -n \log \sigma - \frac{n}{2} \log(2\pi) - \sum_i (x_i - \mu)^2 / (2\sigma^2)$. Both slopes vanish at $\hat{\mu} = \bar{x}$ and $\hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2$, with n , not $n-1$. For 20 draws from $\mathcal{N}(5, 2^2)$: $\hat{\mu} = 4.63$, $\hat{\sigma} = 1.70$ (1.75 with $n-1$). The truth scores -39.82 , below the peak's -39.01 : on its own sample the MLE beats the truth. `np.std` divides by n , `pandas.std()` by $n-1$.

8 Poisson and exponential: the rate comes from the mean



Poisson counts: $\ell(\lambda) = \sum_i (k_i \log \lambda - \lambda - \log k_i!)$ has slope $\sum_i k_i / \lambda - n$, zero at $\hat{\lambda} = \bar{k}$: 46 events in 12 hours give 3.83 an hour. Exponential waits: $\ell = n \log \lambda - \lambda \sum_i x_i$, so $\hat{\lambda} = 1/\bar{x} = 0.917$ a minute. For a Bernoulli, $\hat{p}$ is the share of ones.

9 Compare models by log-likelihood on the same data



Fit each model, then compare ℓ on the same points: higher wins. On 200 simulated positive values a gamma scores -360.4 , a normal -388.5 (4.3% of its mass is below 0), an exponential -412.5 . The gamma's shape has no closed form; we searched a grid of k with $\theta = \bar{x}/k$. A model that contains another (the gamma at $k=1$ is the exponential) never scores lower on its own data, so also score new data. ℓ on different data sets does not compare.

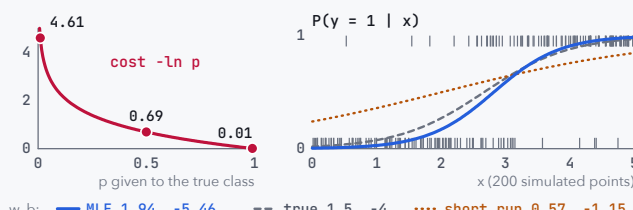
10 Without a closed form, climb in small steps

Libraries minimise the mean negative log-likelihood (NLL) $-\frac{1}{n} \sum_i \log p(x_i | \theta)$ by gradient steps. The same loop in NumPy for the normal of panel 7 (the dotted path there), with $s = \log \sigma$ so σ stays positive:

```
mu, s = 3.0, np.log(3.5) # start
for step in range(300):
    sig = np.exp(s)
    z = (x - mu) / sig
    mu += 0.2 * (z / sig).mean()
    s += 0.2 * (z**2 - 1).mean()
mu, np.exp(s) # 4.6336, 1.7018
```

The two means are the slopes of ℓ/n ; the NLL falls from 2.399 to 1.9506, the closed-form minimum. In PyTorch the loss is `-d.log_prob(x).mean()` and an optimiser takes the steps.

11 Negative log-likelihood is the cross-entropy loss



A label $y \in \{0, 1\}$ with $p = \sigma(wx + b)$ is a Bernoulli draw: $\log p(y) = y \log p + (1 - y) \log(1 - p)$. Its mean NLL is the binary cross-entropy (BCELoss, or the more stable BCEWithLogitsLoss on $w x + b$). A point costs $-\ln$ of the chance given to its true class. On 200 points from $w = 1.5$, $b = -4$, the MLE has mean NLL 0.318 (truth 0.329, a constant 0.49: 0.693). 1,000 steps of size 0.01 stop at NLL 0.476: check that the loss has flattened.

12 Pause and think

3 heads in 3 flips: the MLE? $\hat{p} = 1$, so tails "cannot happen". Too few flips.

Data 2, 4, 6 under a normal: $\hat{\mu}$, $\hat{\sigma}^2$? 4 and $8/3$ ($n-1$ gives 4).

Counts 3, 5, 4 under a Poisson: $\hat{\lambda}$? 4.

$\log(e^{-1000} + e^{-1000})$? $-1000 + \ln 2 = -999.31$.

Why `.mean()` and not `.sum()` in the loss? Same minimum; the step size stops growing with n .

The true class gets $p = 0.01$. Its cost? $-\ln 0.01 = 4.61$.

One point outside a uniform's support? $\ell = -\infty$: that model cannot have made the data.

$$L(\theta) = p(\text{data} | \theta) \text{ ranks } \theta \cdot \ell = \sum_i \log p(x_i | \theta) \cdot \text{MLE: } k/n; \bar{x} \text{ and } \hat{\sigma} \text{ with } n; \bar{k}; 1/\bar{x} \cdot \text{NLL} = \text{cross-entropy}$$

Source: the course notebooks on continuous distributions (fitting by minimising `-log_prob(x).mean()`; $\mathcal{N}(0, 1)$ at 17 has log probability -145.4), on discrete distributions and CDFs, and on generative logistic regression (fitting with a cross-entropy loss), nipunbatra.github.io/psdv-teaching. The coin and all data sets are our own, simulated with seed 0 by the sheet's figure script. Next: the random vectors and PCA sheet.